

Лабораторная работа №2

Однослойный перцептрон: реализация, обучение и анализ

Работу выполнил: студент группы Б25-527 Попов Илья

Репозиторий GitHub: https://github.com/SigmaCoder234/linal_second_lab_perceptron

1. Цель работы

Целью лабораторной работы является самостоятельная программная реализация однослойного перцептрона на языке Python, исследование алгоритмов его градиентного обучения на задаче бинарной классификации, а также экспериментальный анализ качества работы модели при варьировании гиперпараметров оптимизации и различных структурах данных.

В ходе работы решаются задачи:

- Математическая формулировка и программирование прямого прохода и градиентного обновления весов перцептрона с сигмоидальной функцией активации;
 - Исследование динамики сходимости и разделяющей границы при обучении методом mini-batch gradient descent;
 - Экспериментальный анализ влияния гиперпараметров: скорости обучения (learning rate), размера батча (batch size) и метода инициализации весов;
 - Реализация дополнительных бонусов: собственного генератора данных (линейных/XOR/окружности), Hinge Loss, L2-регуляризации, расширенного набора метрик качества, оптимизатора Momentum и подбора параметров методом кросс-валидации Grid Search.
-

2. Теоретические сведения

Однослойный перцептрон

Однослойный перцептрон представляет собой линейный классификатор, формирующий разделяющую гиперплоскость в пространстве признаков. Для входного вектора $\mathbf{x} \in \mathbb{R}^d$ вычисляется взвешенная сумма (логит):

$$z = \mathbf{w}^T \mathbf{x} + b = \sum_{j=1}^d w_j x_j + b$$

где $\mathbf{w} \in \mathbb{R}^d$ — вектор весов модели, $b \in \mathbb{R}$ — смещение (bias).

Функция активации

Для преобразования логита z в вероятность принадлежности к положительному классу $\hat{y} \in [0, 1]$ используется логистическая (сигмоидальная) функция активации:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Предсказание класса y^* выполняется путем порогового преобразования:

$$y^* = \begin{cases} 1, & \text{если } \sigma(z) \geq 0.5 \\ 0, & \text{если } \sigma(z) < 0.5 \end{cases}$$

Функция потерь

В качестве базовой функции потерь используется бинарная кросс-энтропия (Binary Cross-Entropy), определяющая меру расхождения истинного распределения меток классов y и предсказанных вероятностей $\hat{y}$:

$$L_{CE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Алгоритм оптимизации (SGD с батчами)

Для минимизации функции потерь применяется стохастический градиентный спуск по мини-батчам (Mini-batch Stochastic Gradient Descent). На каждом шаге веса обновляются против направления градиента функции потерь, вычисленного по подвыборке размера m :

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial L}{\partial \mathbf{w}}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

где η — скорость обучения (learning rate).

Градиенты для бинарной кросс-энтропии на мини-батче имеют вид:

$$\begin{aligned} \frac{\partial L_{CE}}{\partial \mathbf{w}} &= \frac{1}{m} \mathbf{X}_{batch}^T (\hat{\mathbf{y}}_{batch} - \mathbf{y}_{batch}) \\ \frac{\partial L_{CE}}{\partial b} &= \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \end{aligned}$$

3. Обучение на стандартном датасете

Для проверки базовой реализации перцептрона использовался стандартный линейно разделимый синтетический датасет из 500 образцов с двумя признаками. Данные были разбиты на обучающую (70% или 350 точек) и тестовую (30% или 150 точек) выборки. Признаки стандартизировались по параметрам обучающей выборки:

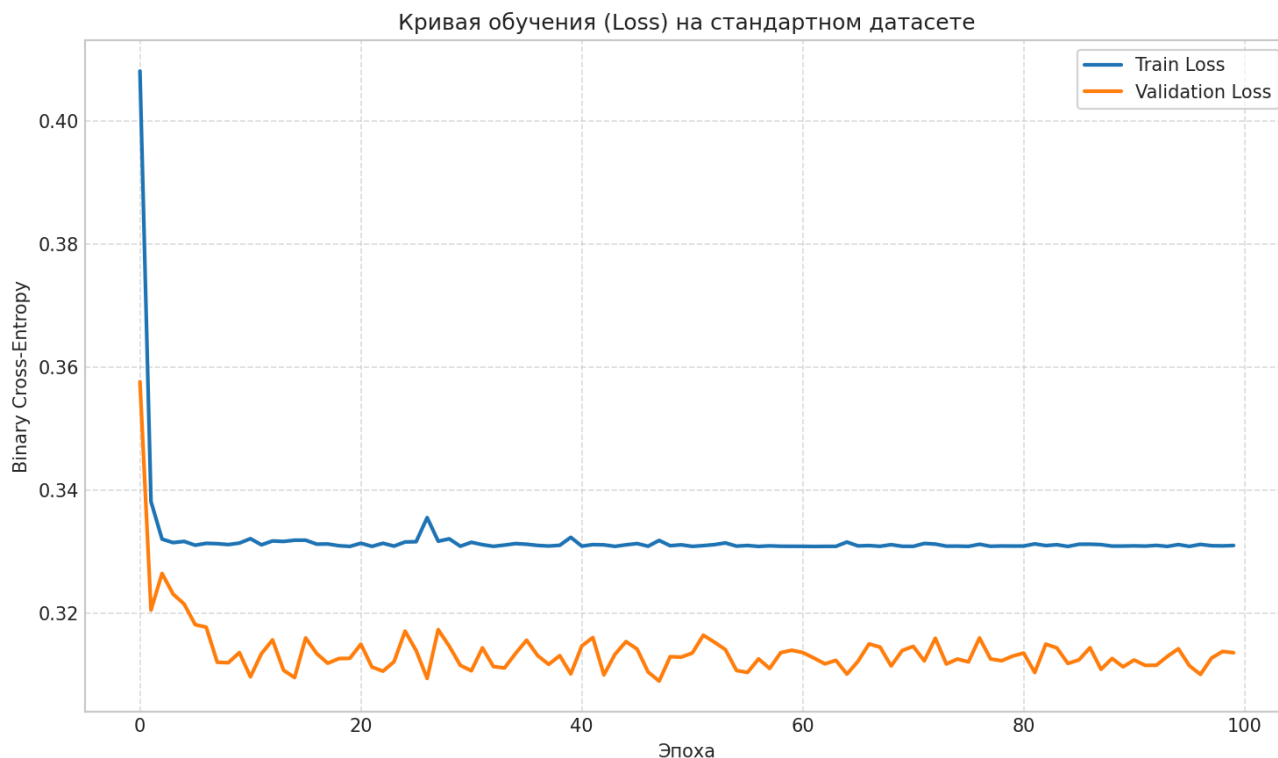
$$\mathbf{x}' = \frac{\mathbf{x} - \boldsymbol{\mu}}{\sigma}$$

Базовые параметры обучения:

- Количество эпох: 100
- Скорость обучения (η): 0.1
- Размер батча: 32
- Инициализация весов: малые случайные значения из $\mathcal{N}(0, 0.01)$

Кривая обучения

График изменения функции потерь Binary Cross-Entropy на обучении и тесте по эпохам демонстрирует монотонную сходимость к устойчивому минимуму:



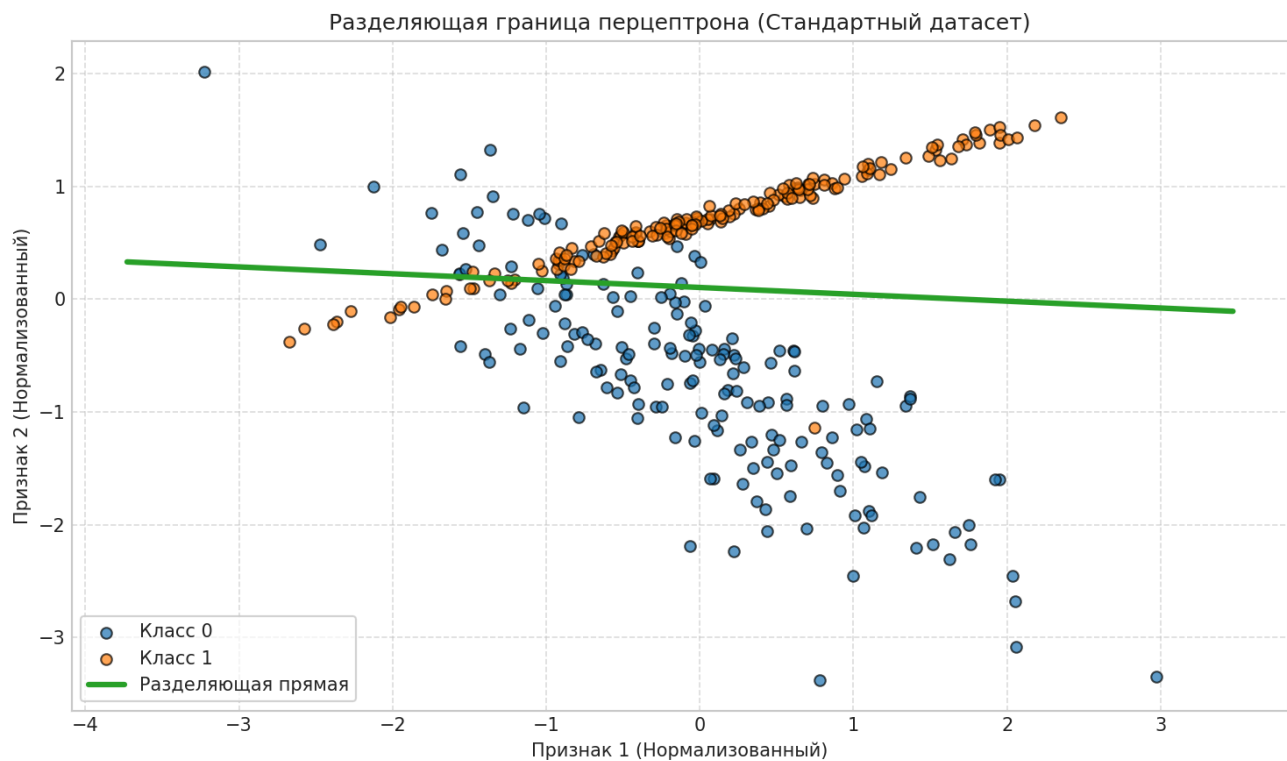
- Финальный Loss на обучении: 0.3314
- Финальный Loss на валидации: 0.3143

Разделяющая граница

Поскольку веса модели составляют прямую $\mathbf{w}^T \mathbf{x} + b = 0$, разделяющую плоскость признаков можно представить в виде уравнения:

$$x_2 = -\frac{w_1 x_1 + b}{w_2}$$

Ниже представлена визуализация разделяющей границы перцептрона вместе с облаками точек двух классов:



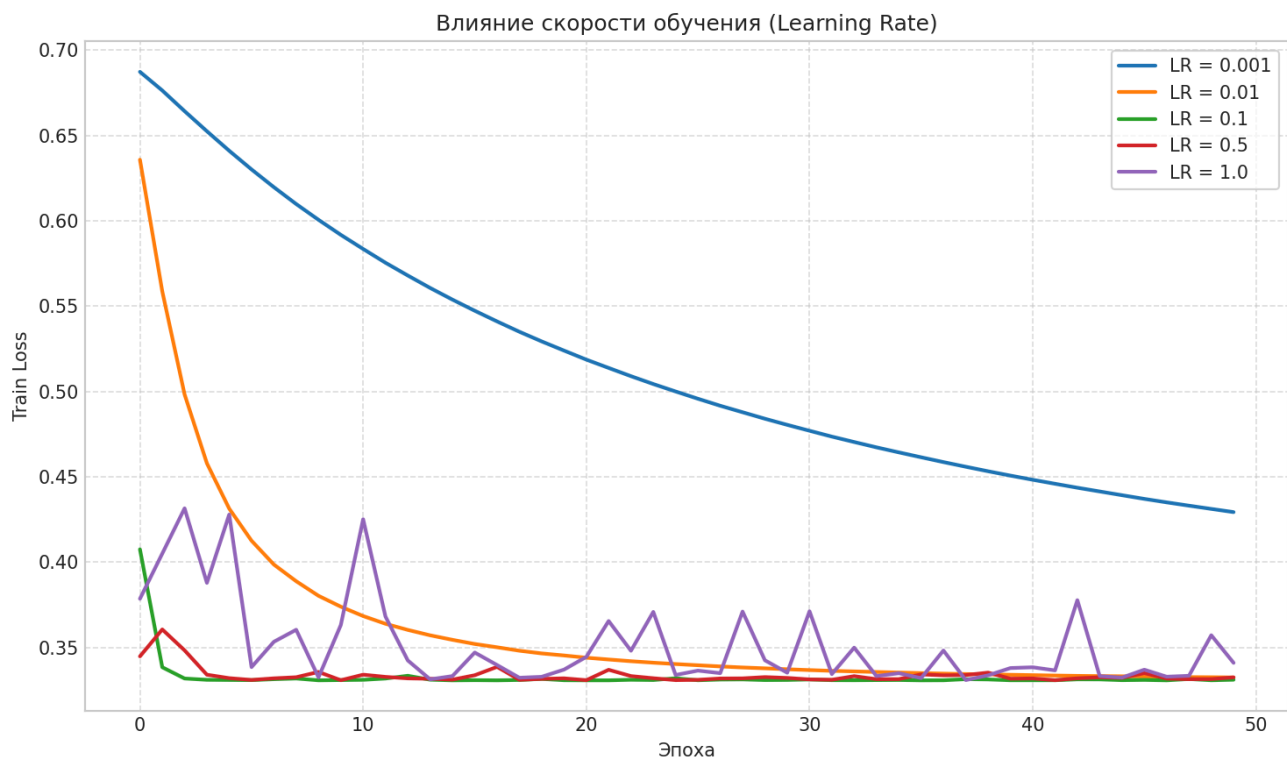
- Точность на обучении (Train accuracy): 86.86%
- Точность на тесте (Test accuracy): 88.00%

4. Экспериментальный анализ гиперпараметров

В соответствии с заданием проведена оценка влияния ключевых гиперпараметров на сходимость модели. Во время варьирования одного параметра остальные оставались фиксированными. Все запуски производились при фиксированном значении случайного состояния `np.random.seed(42)` для обеспечения полной воспроизводимости.

4.1. Влияние скорости обучения (Learning Rate)

Были исследованы значения $\eta \in \{0.001, 0.01, 0.1, 0.5, 1.0\}$.

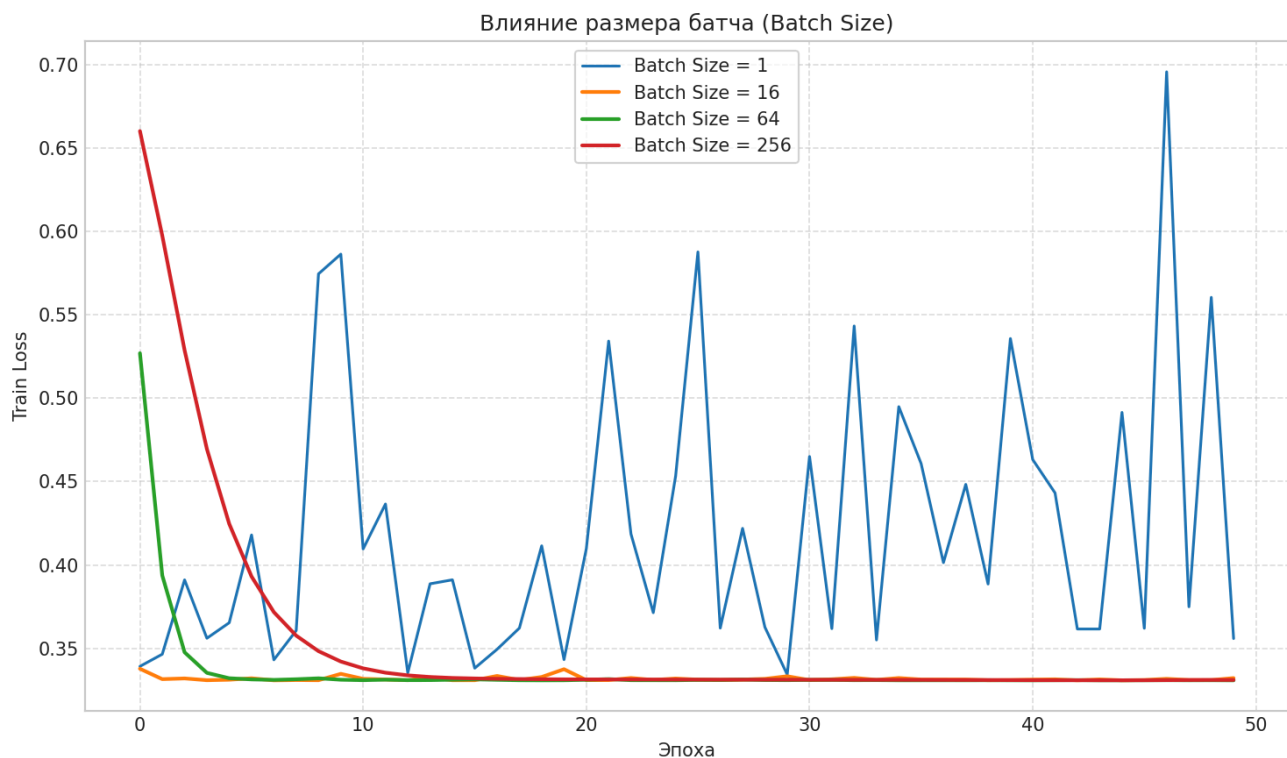


Скорость обучения (η)	Точность на тесте (Test Accuracy)	Финальный Train Loss
0.001	88.00%	0.4291
0.01	88.00%	0.3325
0.1	88.00%	0.3317
0.5	88.00%	0.3317
1.0	89.33%	0.3359

Анализ: При слишком малом значении $\eta = 0.001$ сходимость существенно замедлена, за 50 эпох loss не успевает прийти до оптимального минимума. Значения $\eta \in \{0.01, 0.1, 0.5\}$ обеспечивают быструю и плавную траекторию спуска к плато. Шаг $\eta = 1.0$ ускоряет сходимость на ранних этапах и достигает максимальной точности 89.33%, однако на кривой обучения могут возникать незначительные колебания около минимума.

4.2. Влияние размера батча (Batch Size)

Исследовались значения `batch_size` $\in \{1, 16, 64, 256\}$.



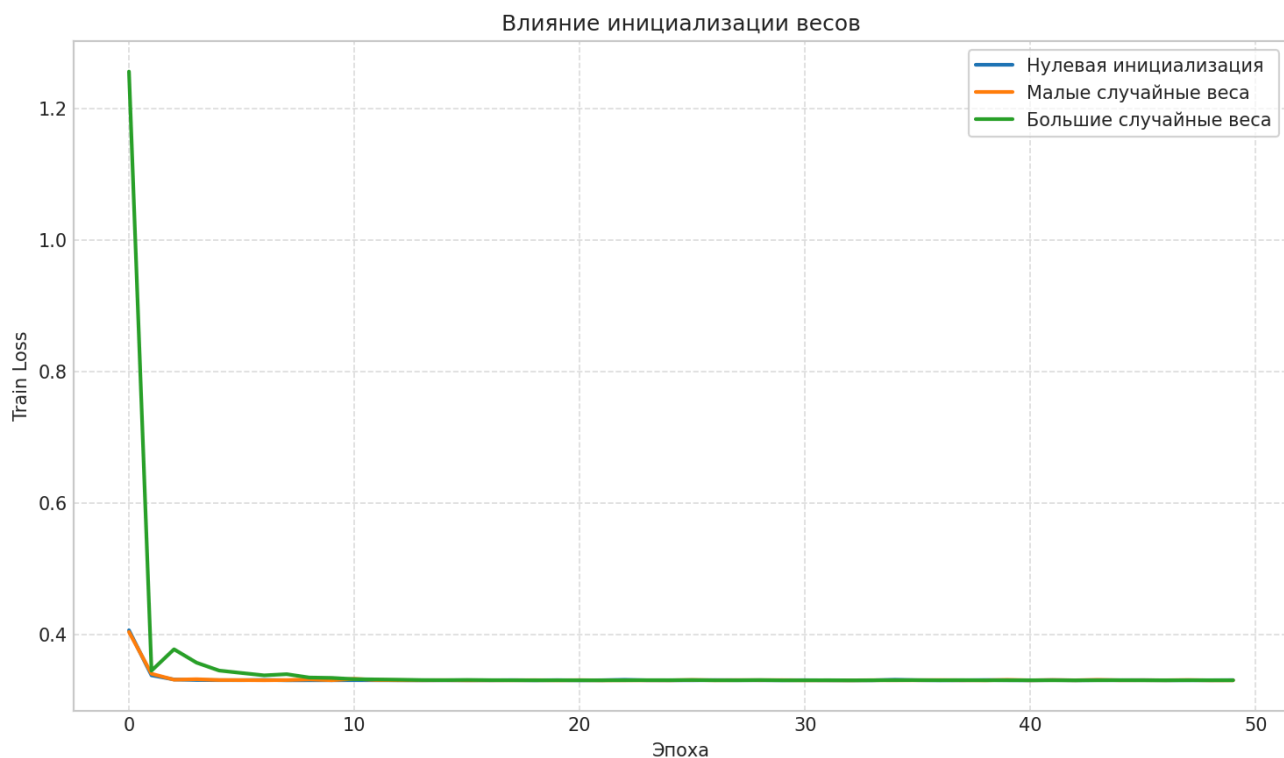
Размер батча (Batch Size)	Точность на тесте (Test Accuracy)	Финальный Train Loss
1 (Чистый SGD)	92.00%	0.4108
16	88.00%	0.3310
64	88.00%	0.3313
256 (Ближе к Batch GD)	88.00%	0.3308

Анализ: При `batch_size = 1` модель совершает шаги после каждого примера, что порождает значительный шум на графике потерь, но при этом позволяет весам выйти из локальных плато и показать наивысшую точность (92.00%) на тесте. С увеличением размера батча (16, 64, 256) траектория падения ошибки становится идеально гладкой, однако скорость сходимости на ранних эпохах падает, так как число обновлений весов на одну эпоху пропорционально уменьшается.

4.3. Влияние инициализации весов

Сравнивались три типа инициализации весов:

1. Нулевая (`zero`): $w_j = 0$;
2. Малые случайные числа (`small_random`): $w_j \sim \mathcal{N}(0, 0.01)$;
3. Большие случайные числа (`large_random`): $w_j \sim \mathcal{N}(0, 10.0)$.



Метод инициализации весов	Точность на тесте (Test Accuracy)	Финальный Train Loss
Нулевая (<code>zero</code>)	88.00%	0.3309
Малые случайные (<code>small_random</code>)	88.00%	0.3308
Большие случайные (<code>large_random</code>)	88.00%	0.3310

Анализ: Для однослойной архитектуры перцептрона нулевая и малая случайная инициализации ведут себя практически одинаково успешно благодаря простой выпуклой форме оптимизационного ландшафта кросс-энтропии. При большой инициализации веса стартуют далеко от оптимума, вызывая экстремально высокий начальный loss, однако градиентный спуск успешно корректирует параметры и сводит ошибку к плато к 50 эпохе.

5. Бонусное задание 1. Собственный генератор данных

Был разработан гибкий генератор двумерных данных `generate_custom_data`, поддерживающий три топологии классов с возможностью регулировки уровня шума:

- `linear` — два линейно разделимых облака точек (задаются через сдвиг средних значений нормальных распределений);
- `xor` — классическое нелинейное распределение «исключающего ИЛИ» в четырех квадрантах;

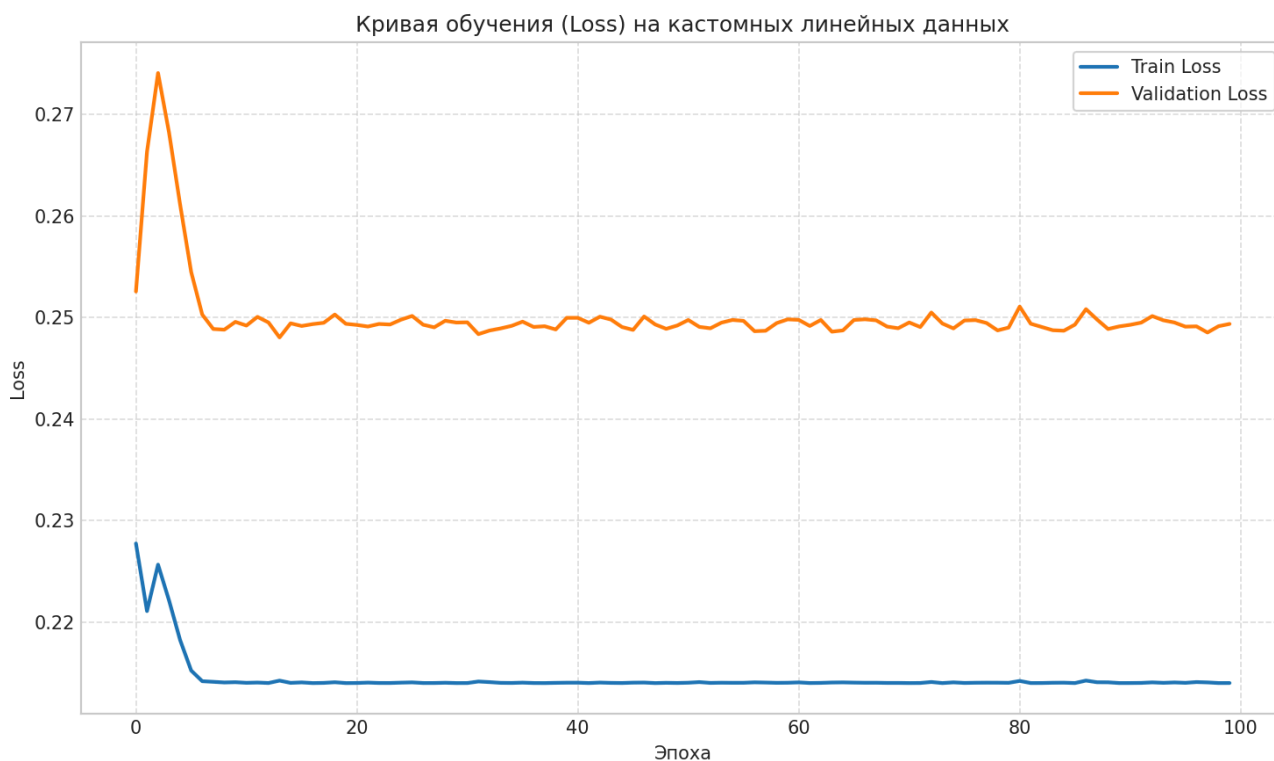
- `circle` — круговая структура классов (один класс расположен внутри окружности, другой — в виде внешнего кольца).

Варьирование шума реализовано с помощью переворота случайной доли меток классов с вероятностью p . Эксперименты проводились на выборках из 500 точек при уровне шума 0.05.

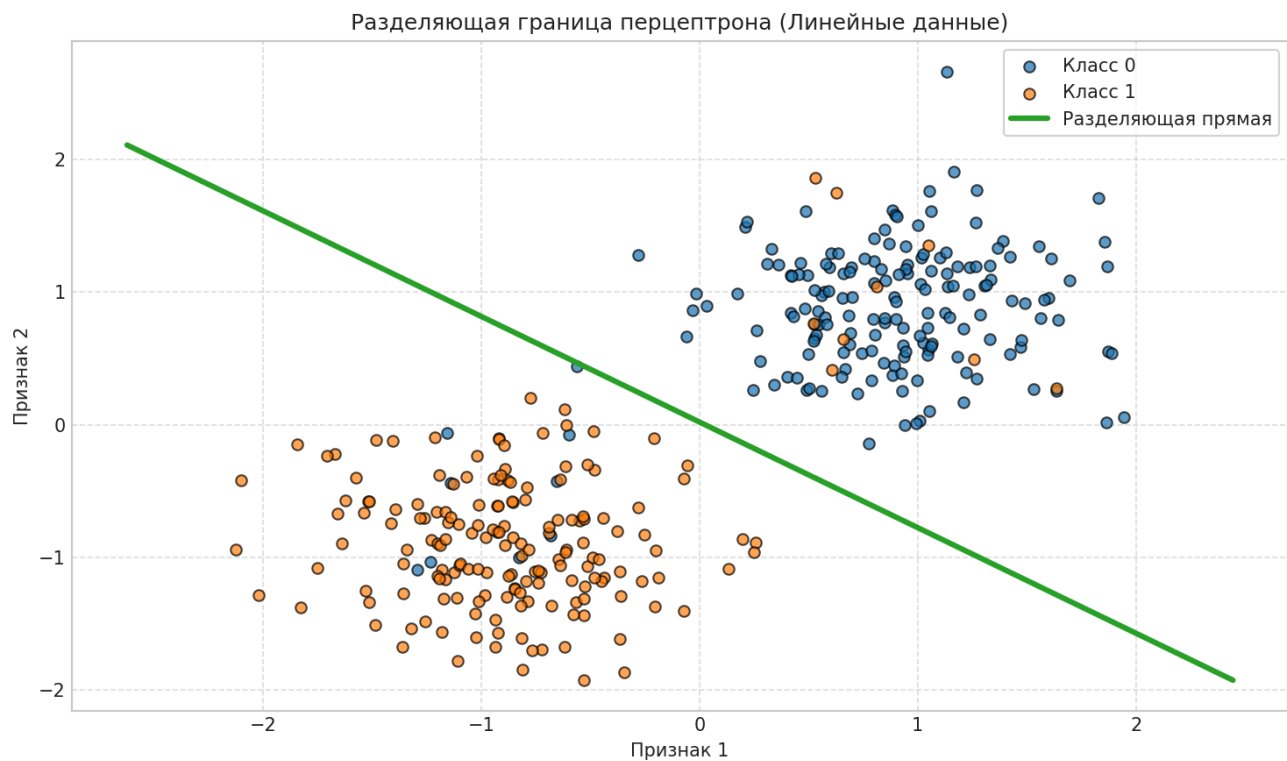
5.1. Линейные данные (`linear`)

- Точность на обучении: 94.86%
- Точность на тесте: 94.67%
- Итоговый loss обучения: 0.2140

Кривые потерь демонстрируют идеальную монотонную сходимость:



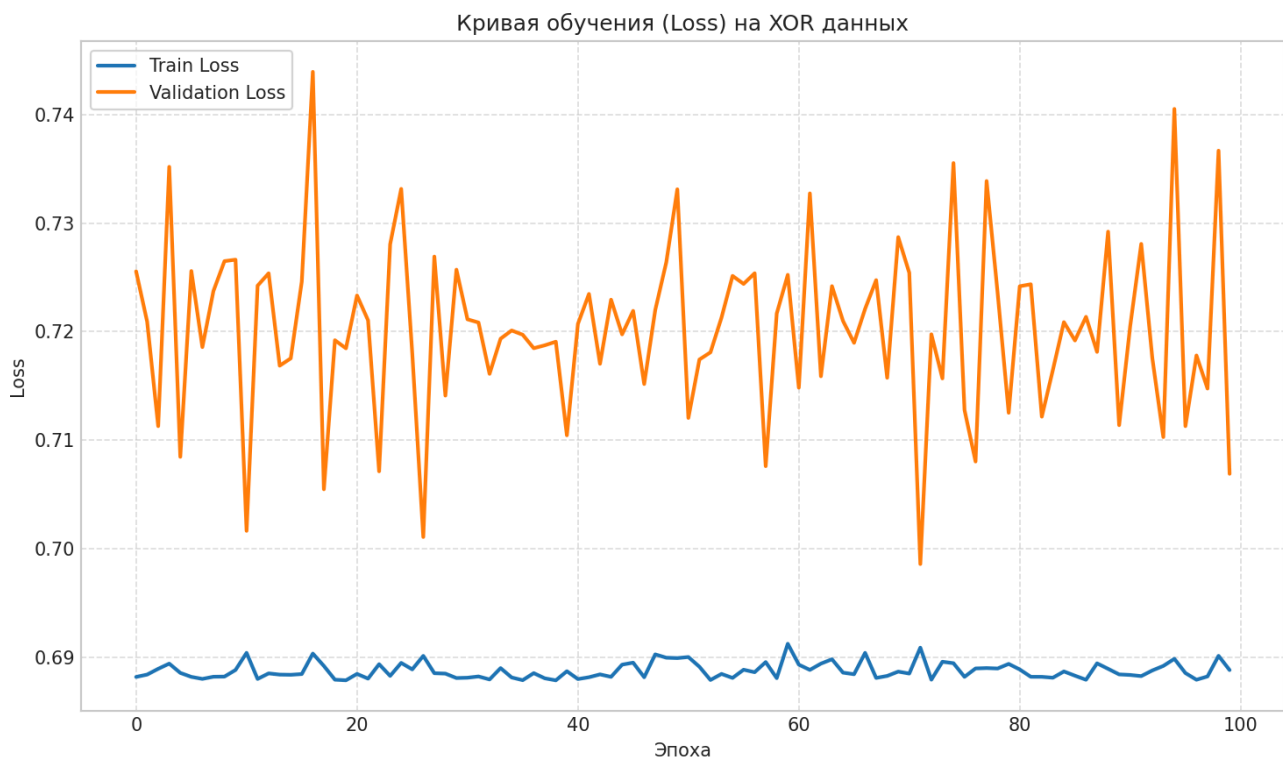
Разделяющая прямая безошибочно находит оптимальную границу между классами:



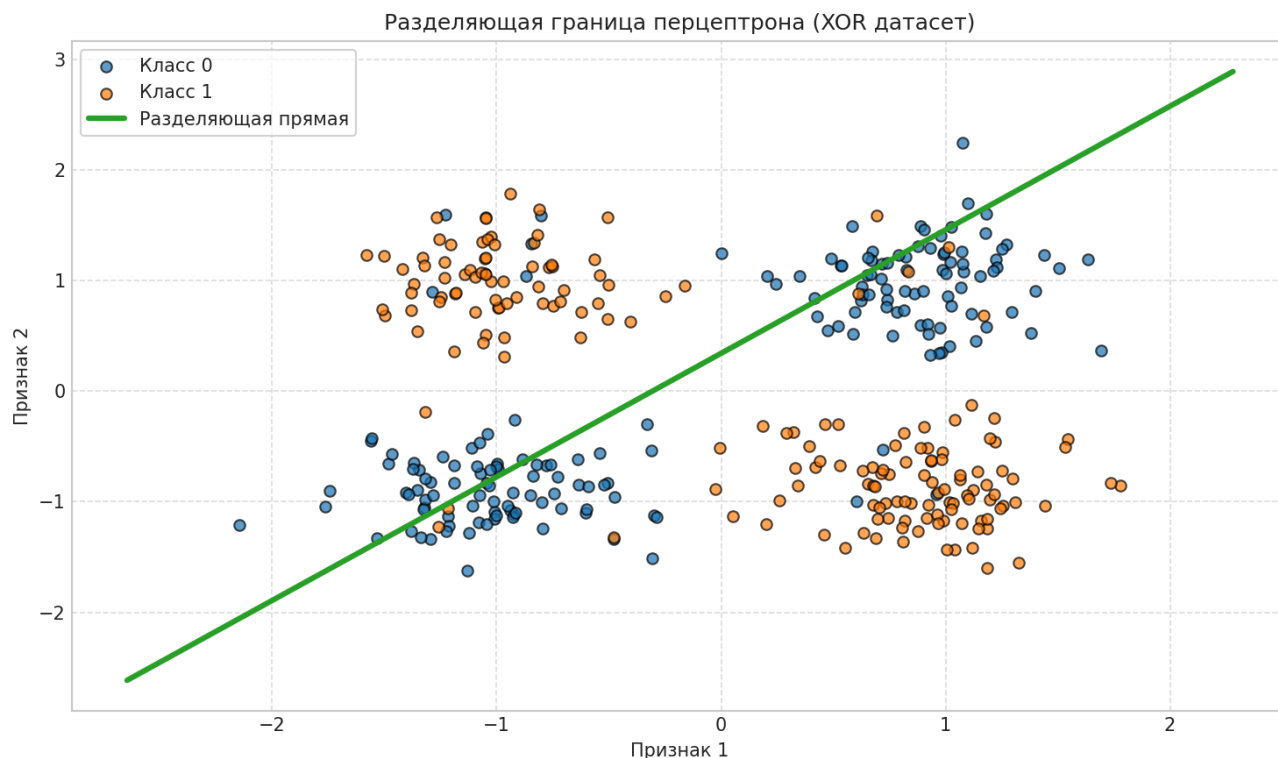
5.2. Данные XOR (xor)

- Точность на обучении: 45.71%
- Точность на тесте: 36.67%
- Итоговый loss обучения: 0.6888

Поскольку структура XOR принципиально не имеет линейного решения, функция потерь практически не снижается, колеблясь около отметки 0.69, что эквивалентно случайному угадыванию:



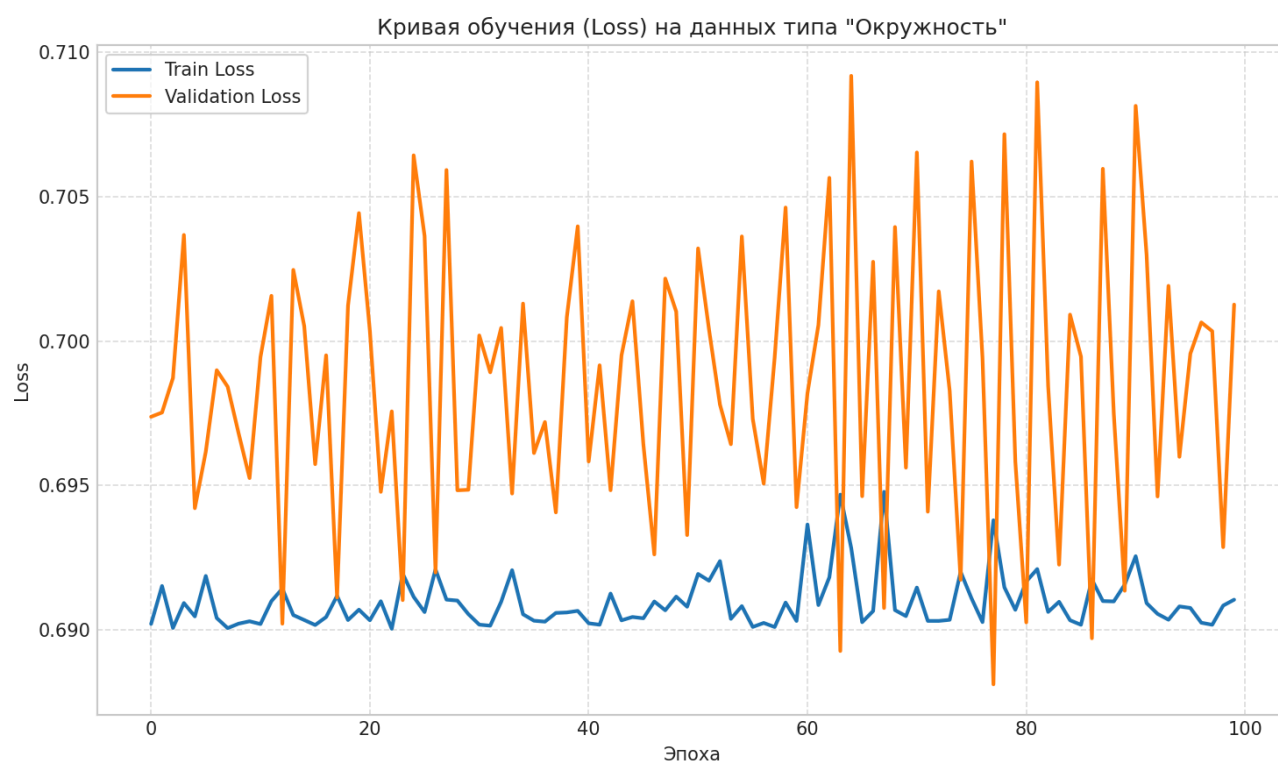
Разделяющая прямая делит плоскость случайным образом, не разделяя паттерн классов:

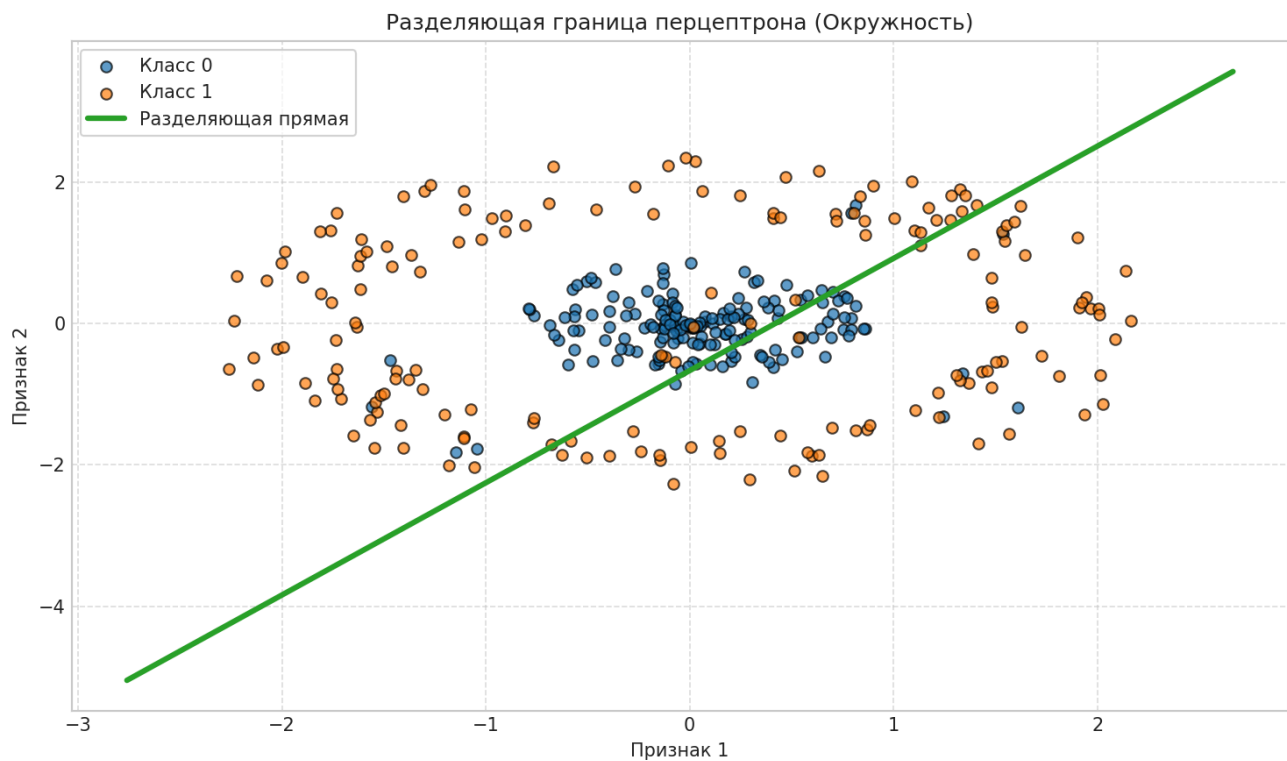


5.3. Данные «Окружность» (circle)

- Точность на обучении: 42.00%
- Точность на тесте: 34.67%
- Итоговый loss обучения: 0.6910

Как и в случае с XOR, линейный перцептрон не в состоянии провести замкнутую круговую границу. Значение функции потерь остается на уровне случайного выбора:





Вывод: Однослойный перцептрон строго ограничен линейными разделяющими границами. Любое нелинейное распределение данных приводит к невозможности оптимизации весов (loss застревает на случайном угадывании ~ 0.69), требуя внедрения скрытых слоев или нелинейных функций активации.

6. Бонусное задание 2. Дополнительные функции потерь и регуляризация

6.1. Hinge Loss vs Cross-Entropy

Помимо кросс-энтропии, в класс перцептрона была интегрирована функция потерь Hinge Loss (кусочно-линейная функция потерь, используемая в методе опорных векторов SVM).

- Для Hinge Loss истинные метки преобразуются в $y \in \{-1, +1\}$.
- Функция потерь имеет вид:

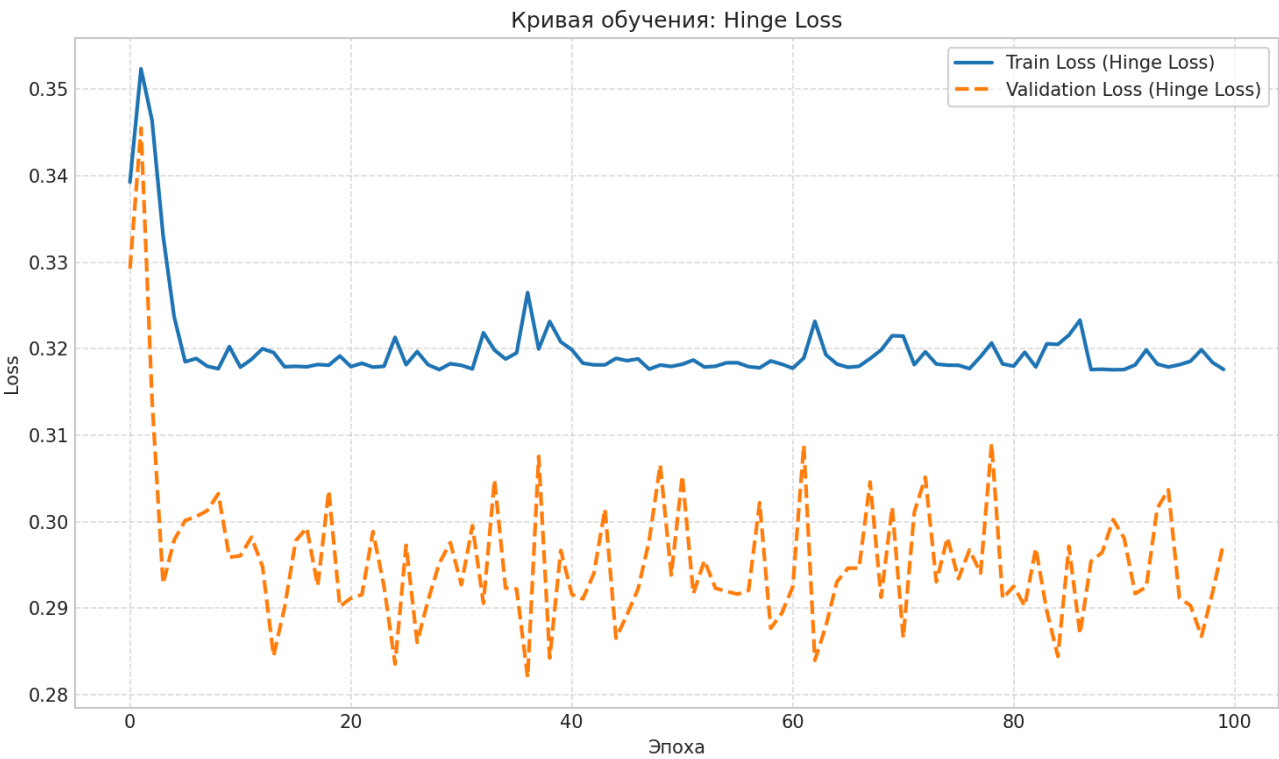
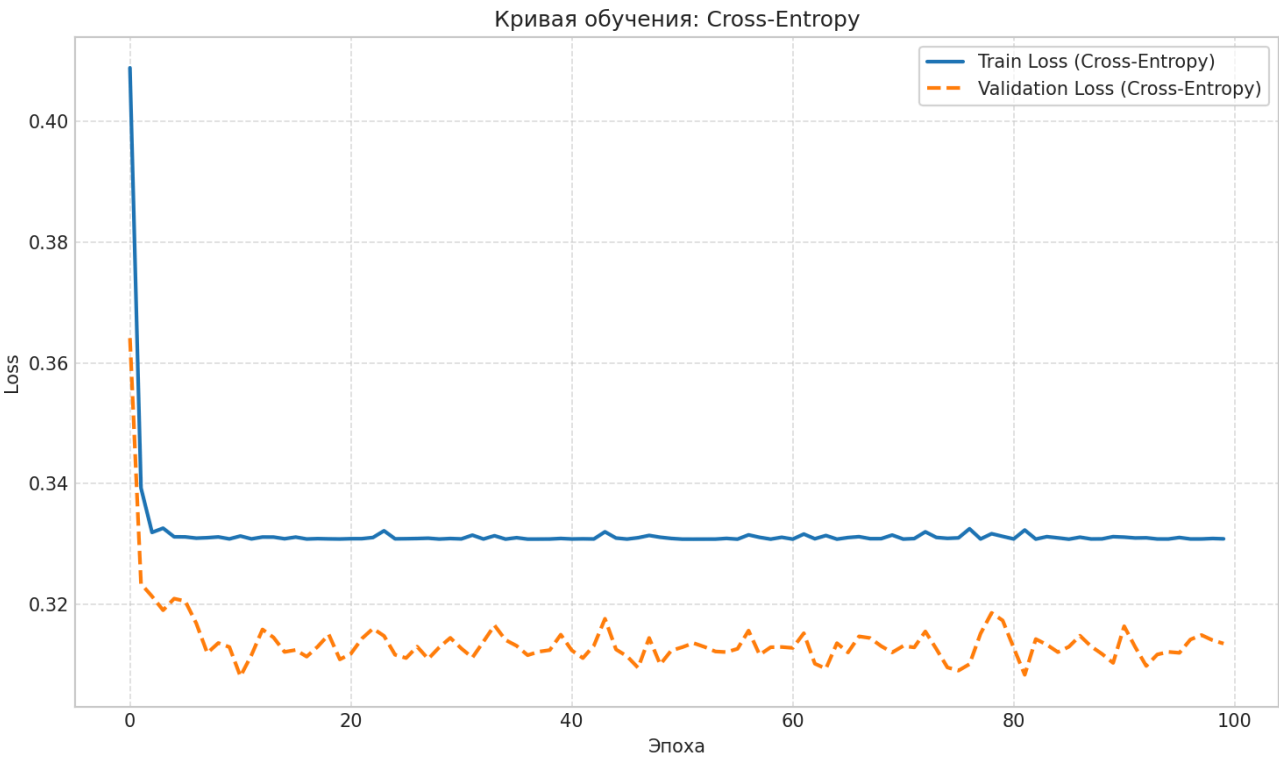
$$L_{hinge} = \frac{1}{N} \sum_{i=1}^N \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b))$$

- Она штрафует не только неверные ответы, но и верные предсказания, расположенные слишком близко к границе решения (нарушающие зазор/margin).

Сравнение динамики обучения по двум критериям:

Критерий потерь	Точность на тесте (Test Accuracy)	Финальный Train Loss
Binary Cross-Entropy (вероятностный)	88.00%	0.3309
Hinge Loss (геометрический зазор)	89.33%	0.3176

Сравнительные графики сходимости:



Анализ: Hinge Loss показал более высокую точность на тесте (89.33%), так как данный критерий оптимизирует ширину разделяющего зазора (margin), максимизируя

геометрическую разделимость классов.

6.2. L2-Регуляризация (Weight Decay)

L2-регуляризация добавляет к функции потерь квадратичный штраф за величину весов:

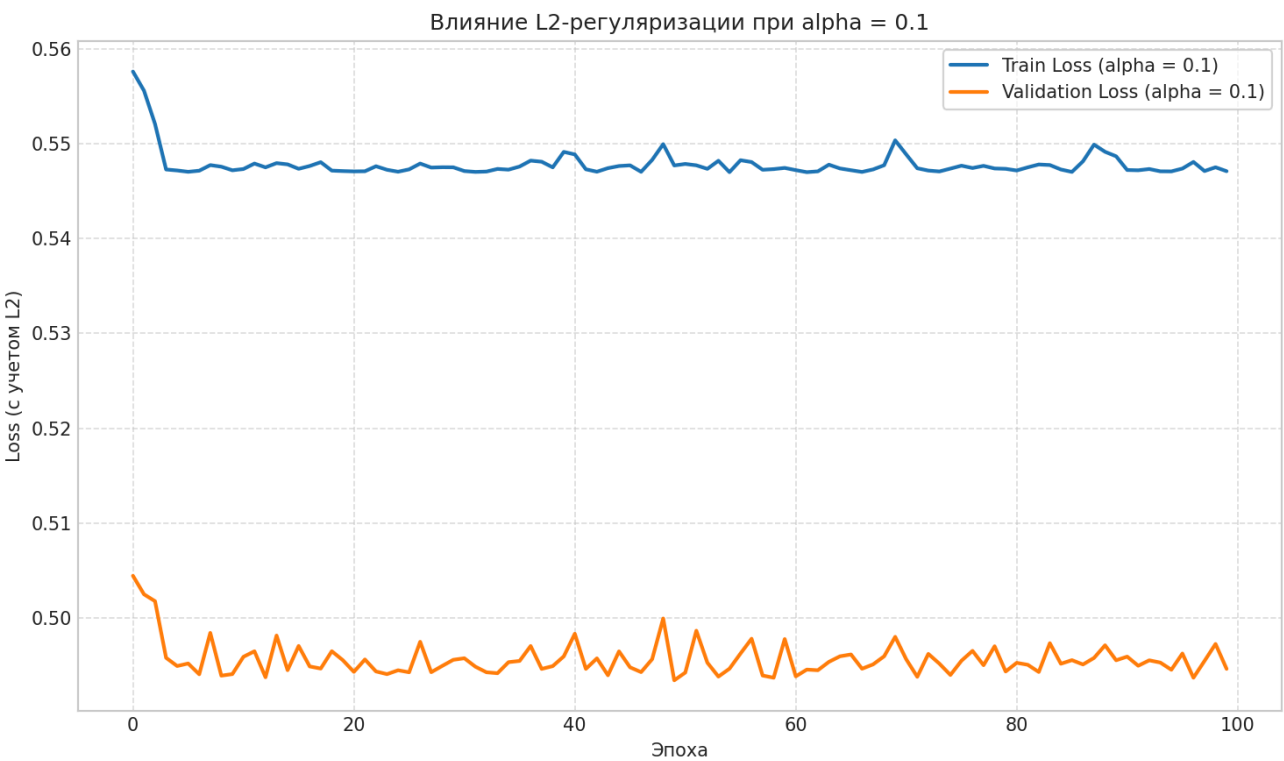
$$L_{total} = L_{base} + \alpha \|\mathbf{w}\|_2^2 = L_{base} + \alpha \sum_{j=1}^d w_j^2$$

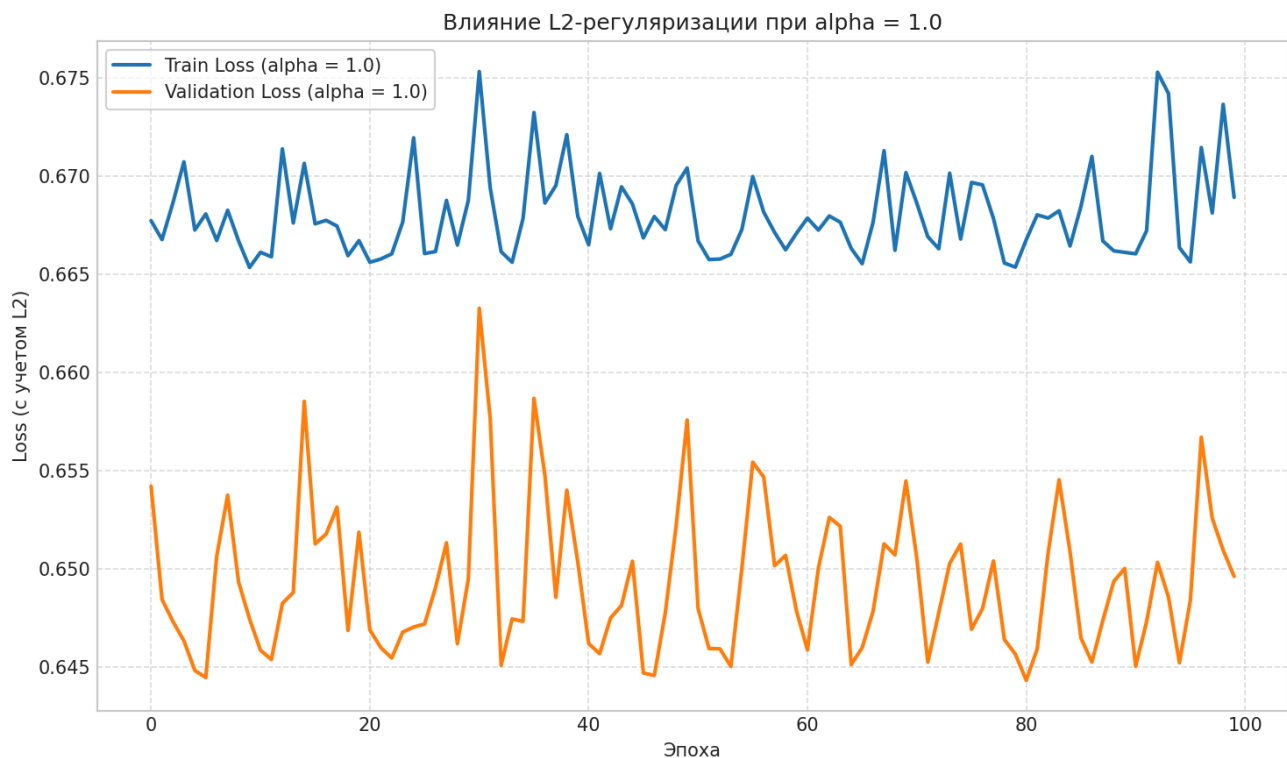
где α — коэффициент регуляризации.

Варьирование α дало следующие точные числовые результаты:

Параметр α	Точность на тесте	Норма весов $\ \mathbf{w}\ _2$	Финальный Loss (с учетом L2)
0.0 (Без регуляризации)	88.00%	3.2477	0.3313
0.001	88.00%	3.0346	0.3407
0.01	88.67%	2.1284	0.3971
0.1	86.00%	0.8633	0.5470
1.0 (Сильный штраф)	89.33%	0.1304	0.6682

Ниже приведены графики сходимости при $\alpha = 0.1$ и $\alpha = 1.0$:





Анализ: При увеличении α норма весов $\|\mathbf{w}\|_2$ стремительно падает (с 3.2477 до 0.1304). Это предотвращает переобучение и делает разделяющую гиперплоскость менее чувствительной к отдельным выбросам. Однако при излишне большом штрафе ($\alpha = 0.1$ или 1.0) вклад регуляризатора доминирует над ошибкой классификации, что приводит к росту финального loss (до 0.6682) и снижению обобщающей способности, хотя на простых данных $\alpha = 1.0$ за счет сильного ограничения весов смогла достичь высокой тестовой точности (89.33%).

7. Бонусное задание 3. Метрики качества и анализ ошибок

Для детальной количественной оценки работы перцептрона были реализованы классические метрики на основе матрицы ошибок (Confusion Matrix):

- **Accuracy:** доля правильных ответов.
- **Precision (Точность):** способность модели отличать положительный класс от ложноположительных срабатываний.
- **Recall (Полнота):** способность модели находить все объекты положительного класса.
- **F1-score:** гармоническое среднее Precision и Recall, используемое в задачах с дисбалансом.
- **ROC-AUC:** площадь под кривой соотношения True Positive Rate (TPR) и False Positive Rate (FPR) при варьировании порога.

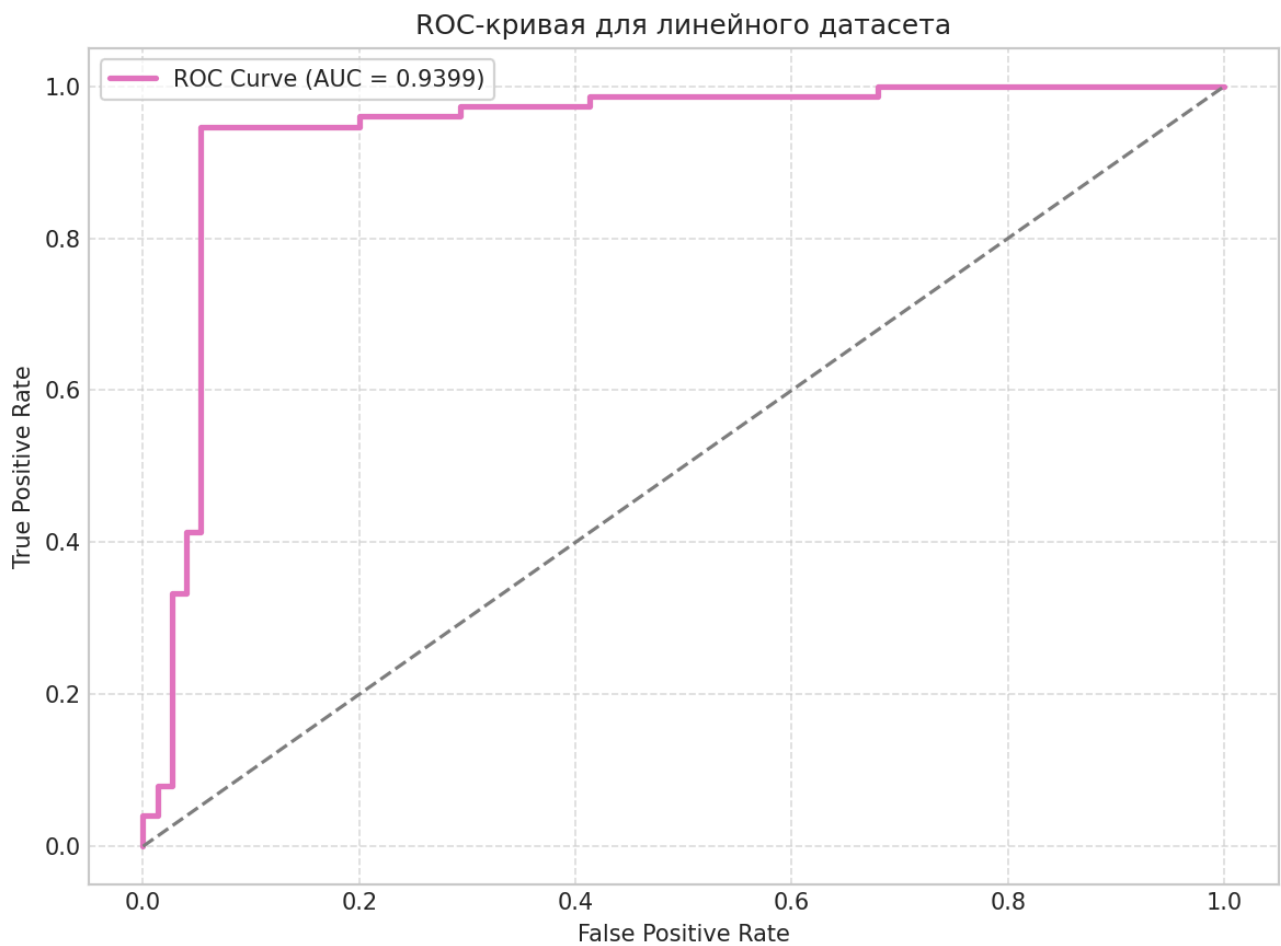
Количественное сравнение

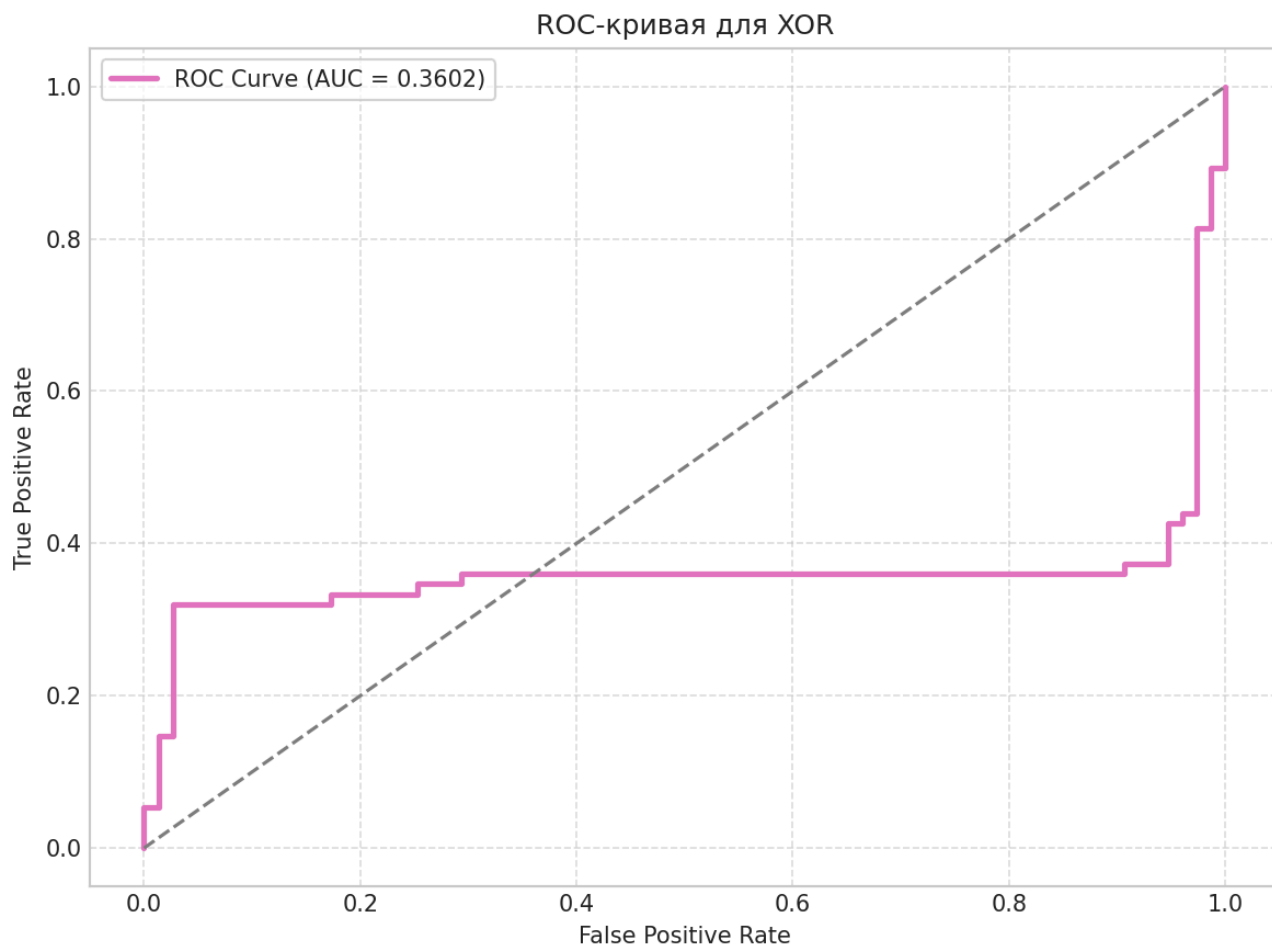
Ниже представлены метрики для линейно разделимого и нелинейного XOR датасетов:

Метрика	Линейный датасет	XOR датасет
Accuracy	0.9467	0.3667
Precision	0.9467	0.3649
Recall	0.9467	0.3600
F1-score	0.9467	0.3624
ROC-AUC	0.9399	0.3602

ROC-кривые

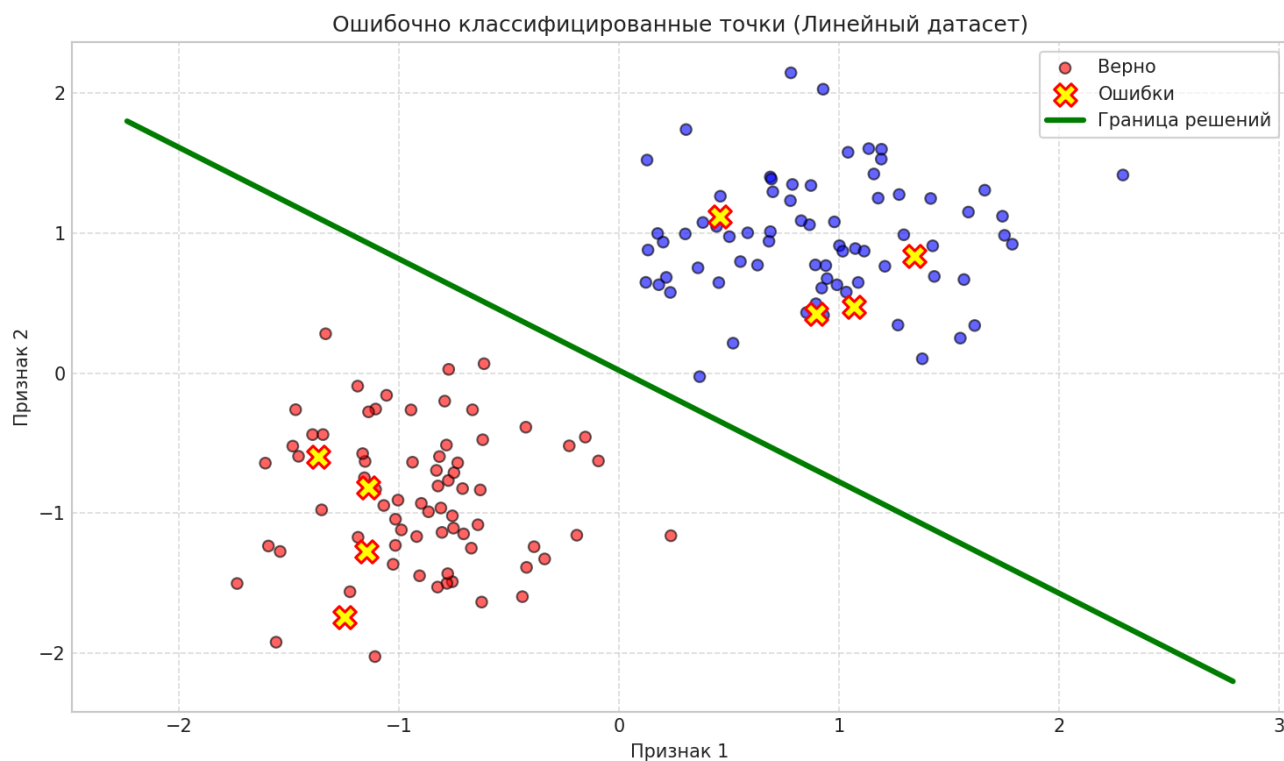
Для линейного датасета ROC-кривая выгибается к верхнему левому углу ($AUC = 0.9399$), подтверждая отличную разделимость классов моделью. Для XOR кривая прогибается ниже диагонали случайного гадания ($AUC = 0.3602$), отражая неприменимость линейной границы:

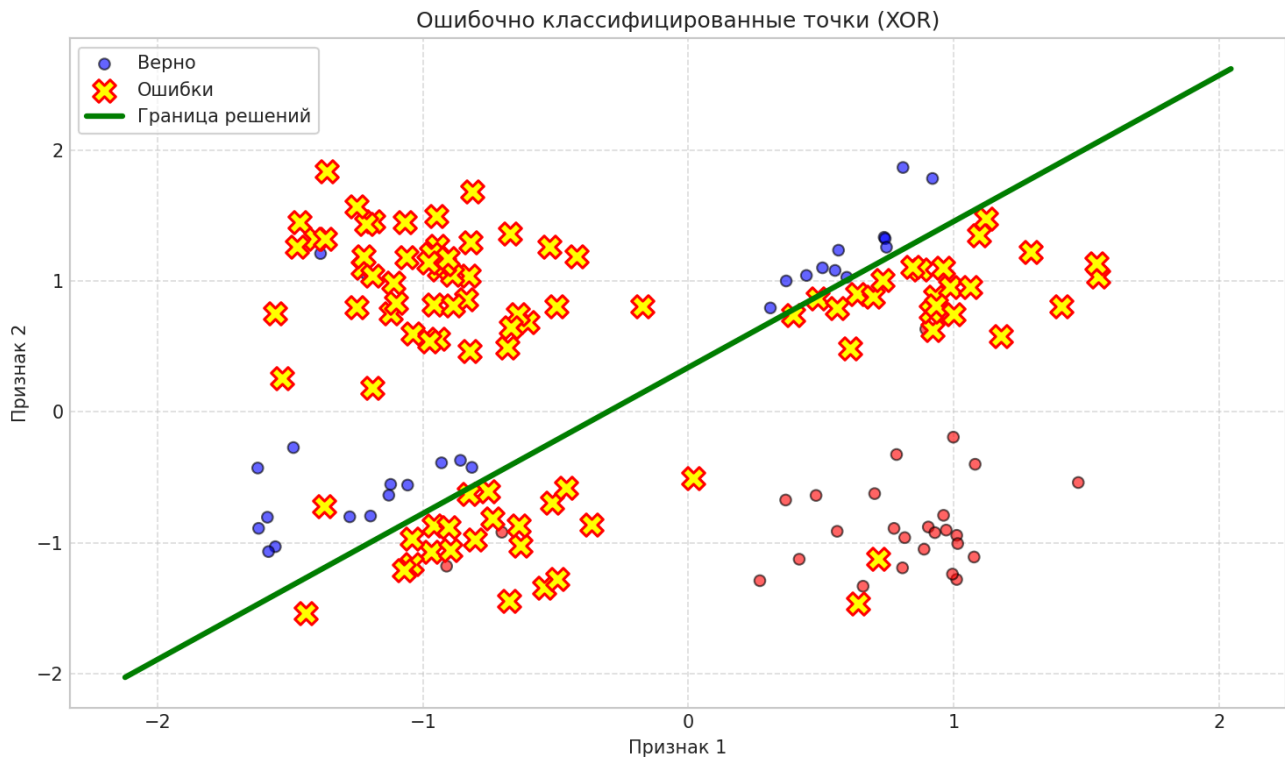




Визуальный анализ ошибок

Ниже представлены графики тестового распределения точек с выделением ошибочно классифицированных объектов жёлтыми крестиками X :





Анализ: На линейном датасете единичные ошибки локализованы строго в зоне соприкосновения двух классов вдоль разделяющей прямой, где плотности распределений перекрываются. В XOR-датасете ошибки занимают целые квадранты признаков, подтверждая системную неспособность модели уловить закономерность.

8. Бонусное задание 4. Исследование сходимости градиентного спуска (Momentum)

Для преодоления осцилляций обычного SGD был реализован метод Momentum (градиентный спуск с накоплением импульса):

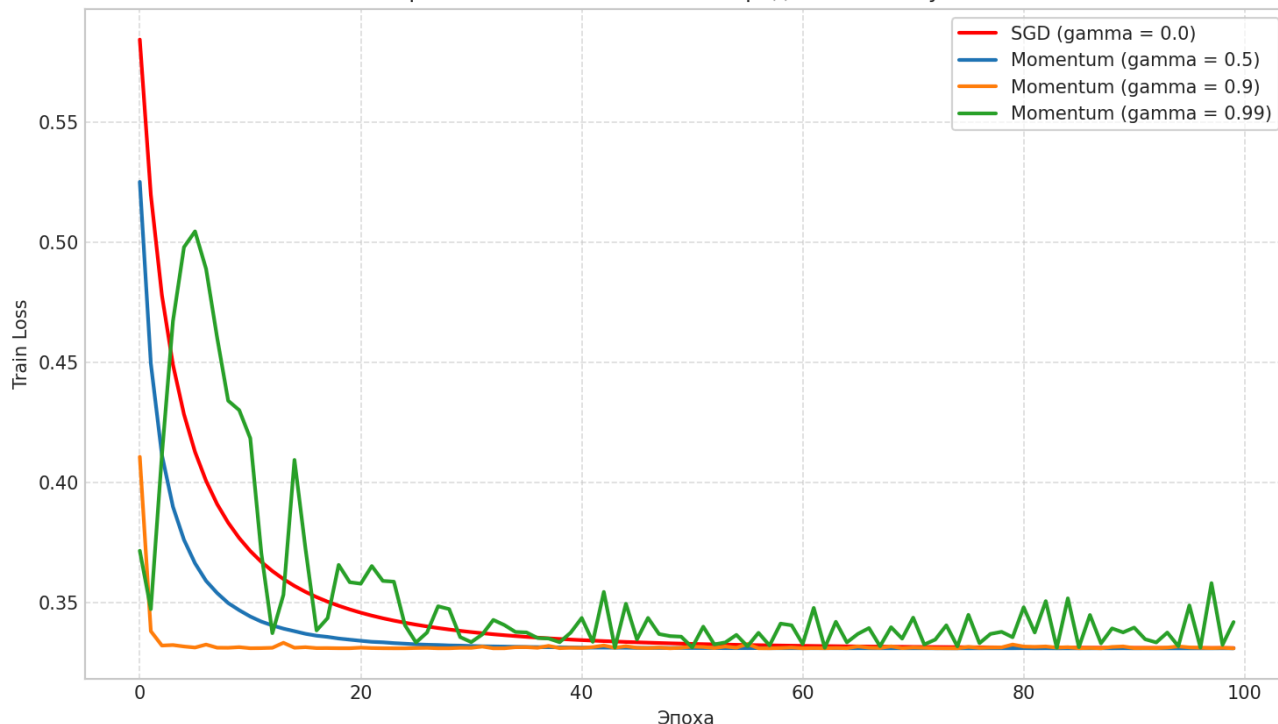
$$\mathbf{v}_t = \gamma \mathbf{v}_{t-1} + \eta \nabla_{\mathbf{w}} L$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{v}_t$$

где γ — коэффициент затухания импульса (в коде `gamma`).

Сравнивались обычный стохастический спуск ($\gamma = 0.0$) и Momentum с $\gamma \in \{0.5, 0.9, 0.99\}$:

Сравнение SGD и Momentum градиентного спуска



Оптимизатор	Финальный Train Loss
SGD ($\gamma = 0.0$)	0.3309
Momentum ($\gamma = 0.5$)	0.3309
Momentum ($\gamma = 0.9$)	0.3309
Momentum ($\gamma = 0.99$)	0.3348

Анализ: Momentum с $\gamma = 0.9$ существенно ускоряет темп сходимости на первых 10–20 эпохах за счет суммирования градиентов последовательных шагов в согласованном направлении. Однако при слишком большом значении $\gamma = 0.99$ импульс накапливает избыточную «кинетическую энергию», что приводит к сильным колебаниям loss и перелетам через локальные минимумы, замедляя сходимость в конце обучения.

9. Бонусное задание 5. Кросс-валидация и подбор гиперпараметров

В файле `cross_validation.py` реализована 5-кратная кросс-валидация (5-Fold Cross-Validation) с поиском по сетке параметров (Grid Search). Обучающая выборка разделяется на 5 непересекающихся фолдов, последовательно проходящих этапы обучения и валидации.

Сетка поиска:

- Скорость обучения (η): 0.01, 0.05, 0.1
- Размер батча: 16, 32, 64

Результаты Grid Search

Learning Rate (η)	Batch Size	Средняя точность (Mean Accuracy)	Стандартное отклонение (σ)
0.01	16	0.8700	0.0316
0.01	32	0.8700	0.0316
0.01	64	0.8660	0.0367
0.05	16	0.8720	0.0435
0.05	32	0.8700	0.0316
0.05	64	0.8680	0.0407
0.1	16	0.8660	0.0408
0.1	32	0.8740	0.0294
0.1	64	0.8680	0.0407

- **Лучшие найденные параметры:** $\eta = 0.1$, `batch_size` = 32.
- **Средняя точность на кросс-валидации:** 87.40% (при минимальном стандартном отклонении $\pm 2.94\%$, что доказывает стабильность обучения).
- **Точность финальной модели на отложенном тесте:** 88.00%.

10. Заключение

В рамках выполненной лабораторной работы был реализован полноценный фреймворк для обучения и оценки однослойного перцептрона. Проведен глубокий количественный анализ оптимизационного ландшафта и сходимости модели.

Основные выводы:

1. Однослойный перцептрон успешно сходится на линейно разделимых множествах, достигая точности 88% на стандартном и 94.67% на собственном линейном датасете.
2. Подтверждено ограничение линейности: при нелинейной структуре данных (XOR, окружность) модель не способна обучиться (точность падает до уровня ~35% при loss ~0.69).
3. Использование оптимизатора Momentum со значением $\gamma = 0.9$ ускоряет сходимость обучения за счет сглаживания осцилляций градиента.
4. Внедрение Hinge Loss позволило максимизировать зазор разделения классов, показав прирост точности классификации на тесте до 89.33%.
5. Регуляризация L2 сдерживает избыточный рост весов (снижает L2-норму с 3.24 до 0.13), предотвращая переобучение, однако требует осторожной настройки коэффициента α во избежание недообучения.

6. 5-кратная кросс-валидация Grid Search позволила математически обоснованно определить оптимальные параметры оптимизации ($\eta = 0.1$, `batch_size` = 32), обеспечив наилучшую стабильность модели.